



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Essential Python Modules

Python for Scripting

Luca Sciullo

Dipartimento di Informatica – Scienza e
Ingegneria

Da materiale originale del prof. Marco Di Felice

Modules

- **Simplicity.** Focus on one relatively small portion of the problem.
- **Maintainability.** modifications to a single portion will not produce an impact on other parts of the program.
- **Reusability:** Functionality defined in a single portion can be easily reused.
- **Scoping:** Modules typically define a separate *namespace*, which helps avoid collisions between identifiers in different areas of a program.

Modular programming: breaking a large coding task into separate, smaller, more manageable subtasks or **modules**.

<https://realpython.com/python-modules-packages/#python-packages>

Modules

A **Python module** is a file containing definitions and statements.

- Every module has a name equal to the file name.

mymodule → mymodule.py

- Definitions from a module can be **imported** into other modules through the **import** statement (code reuse).
- Every module uses its **own namespace** (no variable definition conflicts with other Python script importing that modules).
- To speed up loading modules, Python caches the **compiled version** of each module in the `__pycache__` directory.

Modules

```
def isEven(num):  
    if (num % 2 == 0):  
        return True  
    else:  
        return False
```

mymod.py

```
import mymod as mmod  
  
if (mmod.isEven(5) == True):  
    print("%d is even" %(num))  
else:  
    print("%d is odd" %(num))
```

client.py

Modules

```
def isEven(num):  
    if (num % 2 == 0):  
        return True  
    else:  
        return False
```

mymod.py

```
from mymod import isEven
```

client.py

```
If (isEven(5) == True):  
    print("%d is even" %(num))  
else:  
    print("%d is odd" %(num))
```

Modules

In the previous example, the Python interpreter **searches for a module** named `mymod` in the following folders:

- The directory from which the input script is run
- The list of directories contained in the `PYTHONPATH` variable, if set
- An installation-dependent list of directories given by `sys.path` variable

```
import sys  
print(sys.path)
```

```
['/Users/marcodifelice/Documents/Coding_workspaces/Jupyter/DataLogic',  
, '/Users/marcodifelice/miniforge3/envs/mlp/lib/python38.zip',  
, '/Users/marcodifelice/miniforge3/envs/mlp/lib/python3.8']
```

Modules

Useful built-in operations for modules:

- `__name__` contains the name of the module
- `dir()` returns the list of defined names in the module
- `hasattr(module, name)` returns true/false whether the name exists or not in the module namespace

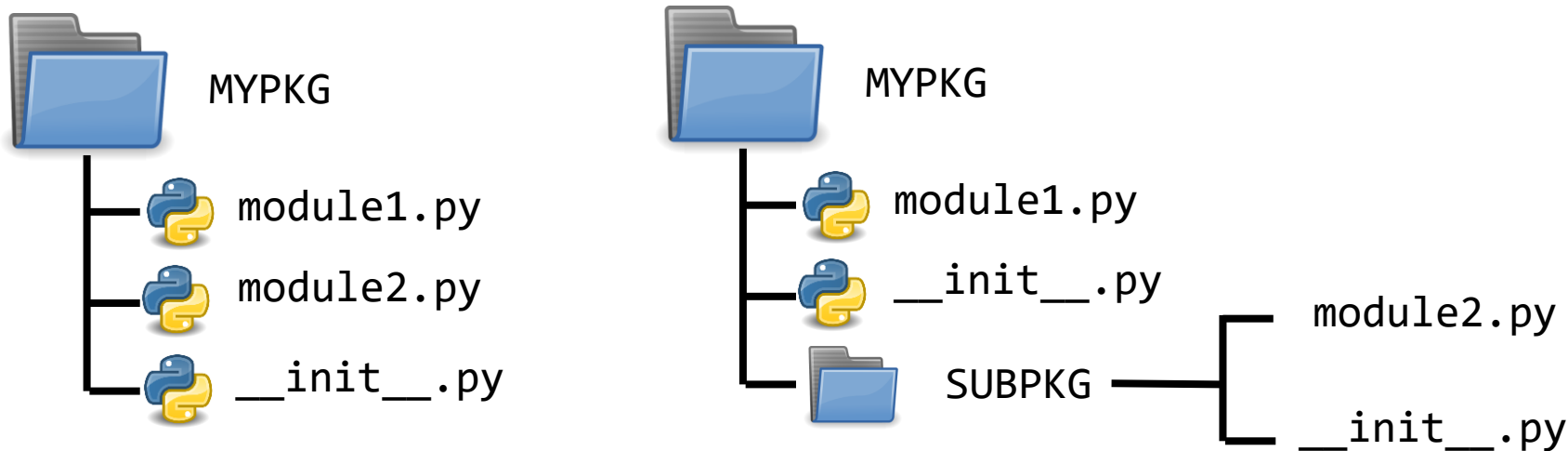
```
x = 3
def hello():
    print("hello")
print(dir())
```

```
['x', 'hello', ...]
```

Modules

A **Python package** is a way of structuring modules related to the same domain/functionality.

- The package consists of a directory containing one or more modules.
- The directory must contain a file named `__init__.py`.
- This file contains the initialization code (can be left empty).



Modules

Standard distribution of Python includes a large number of modules, generally known as **built-in** modules.

- argparse module
- invoke module
- pathlib module
- os module
- sys module
- time module
- datetime module
- math module
- random module
- pickle module

PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS

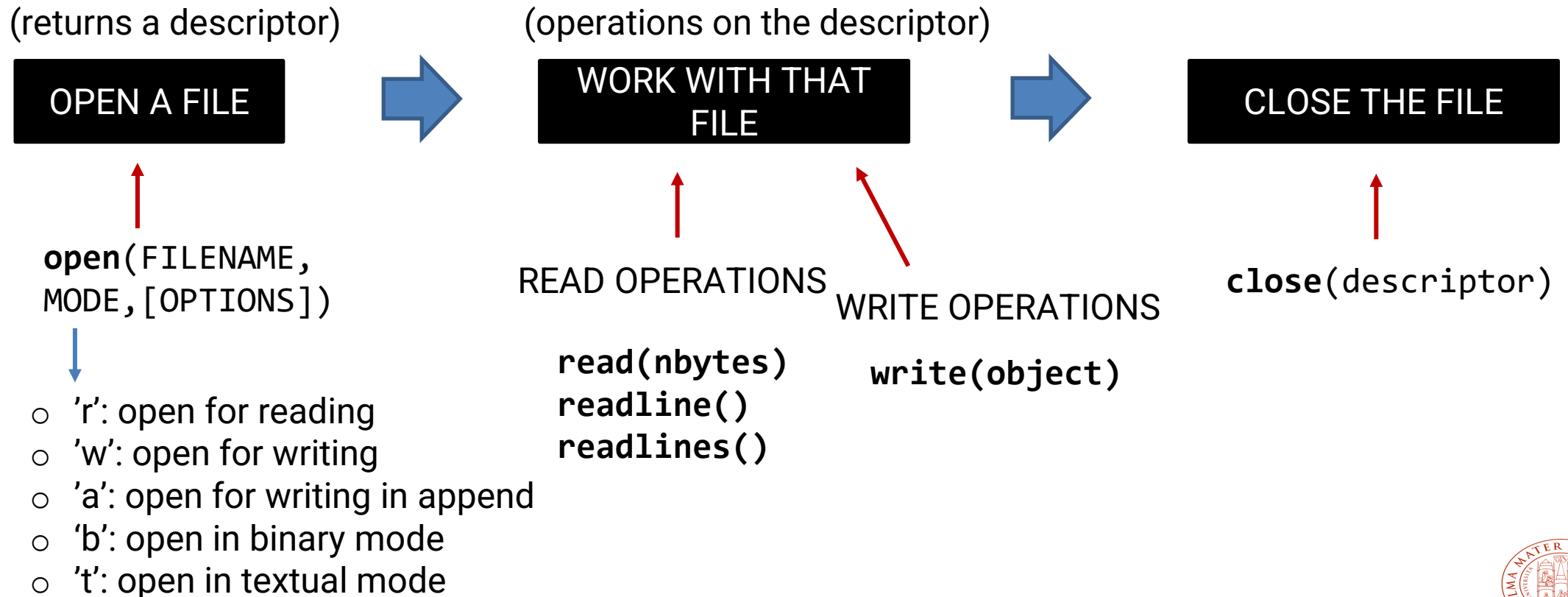
Modules

- **File-system operations**
 - **Modules presented:** **std library, os, pathlib**
- CLI & process management
 - Modules presented: sys, argparse, invoke, subprocess
- Time operations
 - Modules presented: time, datetime
- Math operations
 - Modules presented: math, random
- Data structures operations
 - Modules presented: pickle

**PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS**

Python File I/O

Basic file operations are provided by the built-in Python module



Python File I/O

Creates a new file (names.txt) in the current working directory

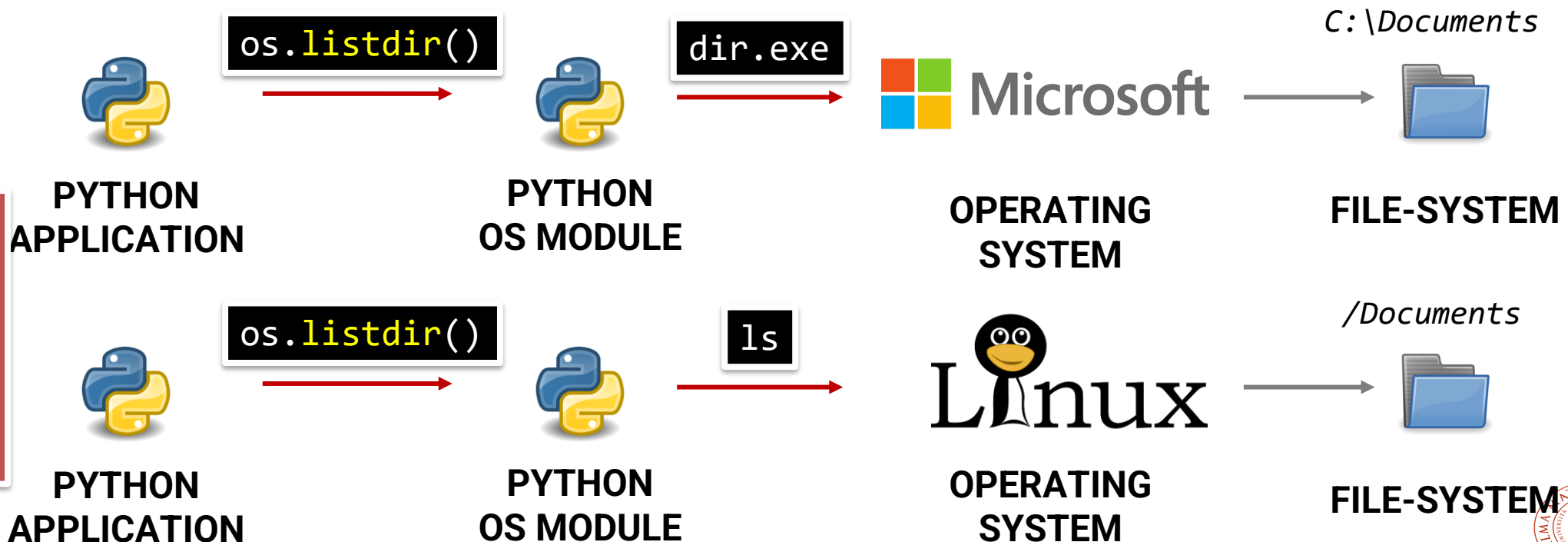
```
f = open("names.txt", "w")  
f.write("Michele Rossi \n")  
f.write("Daniela Bianchi \n")  
f.close()
```

Open the file for reading

```
f = open("names.txt", "r")  
for line in f.readlines():  
    print(line)  
f.close()
```

OS Module

OS module: it provides functions for interacting with the Operating System (OS) and with the file system, abstracting from the OS type.



Example:
Listing a
directory
content
with the
listdir
function of the
os module

OS Module

- It includes many built-in functions for process management, I/O and file-system functionalities (e.g. create a directory).
- **os.name**: returns the name of the OS module it imports.
- **os.environ**: provides access to the environment variables of the system
- **os.system**: allows users to run a OS command in the Python script
- All functions in this module raise an **OSError** in the case of invalid file names or wrong parameters.

OS module: it provides functions for interacting with the Operating System (OS) and with the file system, abstracting from the OS type

OS Module

File system functions

<code>os.getcwd()</code>	Get the current working directory (CWD) of the Python script
<code>os.chdir(path)</code>	Change the current working directory (CWD) to path
<code>os.mkdir(path)</code>	Create a directory with name path
<code>os.listdir(path)</code>	List the content of the directory with name path
<code>os.stat(path)</code>	Return a 'stat_result' object which represents the status of specified path
<code>os.remove(path)</code>	Remove an item (file) with name path
<code>os.rmdir(path)</code>	Remove the directory with name path (directory must be empty)
<code>os.rmtree(path)</code>	Remove recursively the directory and all its subdirectories (empty or not)

OS module: it provides functions for interacting with the Operating System (OS) and with the file system, abstracting from the OS type.

OS Module

```
import os

print("Current directory "+os.getcwd())
newDir = "dirTest"
os.mkdir("dirTest")
os.chdir(newDir)
print("Current directory "+os.getcwd())
```

```
$ /home/marco
/home/marco/dirTest
```


OS Module

Process management functions	
<code>os.kill(pid, sig)</code>	Send signal with code sig to the process pid.
<code>os.abort()</code>	Close the current process
<code>os.fork()</code>	Fork a child process
<code>os.nice(increment)</code>	Increment the «priority» of the process for the scheduling
<code>os.system(cmd)</code>	Launches an OS command from the Python script

OS module: it provides functions for interacting with the Operating System (OS) and with the file system, abstracting from the OS type.

OS Module

```
import os

try:
    print("OS Module name: "+os.name)

    #Returns the list of active processes
    return_value = os.system("ps aux")
    print(return_value)

except OSError:
    print("OS error")
```

Pathlib Module

`Pathlib` module: it provides functions for working with filesystem paths with semantics appropriate for different operating systems.

Paths can be structured differently based on the operating system in use



Microsoft

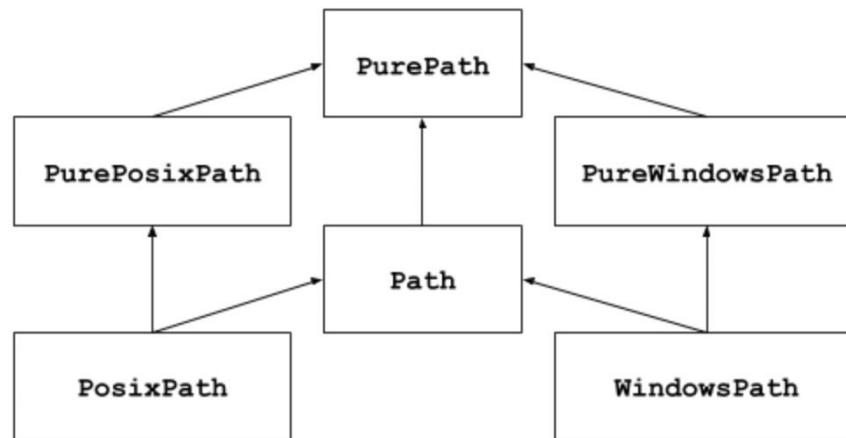
`C:\\Windows\\home\\Documents\\file.txt`



`/home/Documents/file.txt`

Pathlib Module

- Python Paths belong to two classes:
 - **Pure paths:** provide purely *computational operations* without I/O (i.e., without accessing the file-system).
 - **Concrete paths:** in addition to pure paths, *support I/O operations* (e.g., reading the content of a folder).



Path is the class for concrete path:
It inherits methods from `PurePath` class and –in addition- includes methods to deal with the file-system of the target host.

Pathlib Module

- **PurePath(pathsegments):** Path abstract class
- **PurePosixPath(pathsegments):** non-Windows filesystem paths
- **PureWindowsPath(pathsegments):** Windows filesystem paths
- Paths are immutable, support equality check (`PurePath(a) == PurePath(b)`) but do not support ordering.

Few examples of properties/functions for pure paths:

- **PurePath.root:** A string representing the (local or global) root
- **PurePath.parts:** A tuple with to the path's components
- **PurePath.parent:** The parent directory of the current path
- **PurePath.name:** Name of file without directories
- **PurePath.suffix:** The file extension of the path
- **PurePath.is_absolute():** true whether it starts with the global root

Pathlib Module

```
import pathlib  
p = pathlib.PurePath("/home/marco/myfile.txt")  
print(p.parts)
```

```
$ ('/', 'home', 'marco', 'myfile.txt')
```

```
print(p.suffix)
```

```
$ .txt
```

```
print(p.parents)
```

```
$ /home/marco
```

Pathlib Module

- **Path(pathsegments):** Concrete abstract class
- **PosixPath(pathsegments):** non-Windows filesystem paths
- **WindowsPath(pathsegments):** Windows filesystem paths
- A ConcretePath supports all operation of the parent's PurePath

Few examples of properties/functions for concrete paths:

- **Path.cwd():** Return the current working directory
- **Path.home():** Returns the user's home directory
- **Path.exists():** Check whether the file exists
- **Path.is_file():** true whether the path is a file, false otherwise
- **Path.iterdir():** returns the list of items in the path
- **Path.glob(pattern):** Searches for paths matching a pattern

Pathlib Module

```
import pathlib
p = pathlib.Path('.')
counterFiles = 0
counterDir = 0
for x in p.iterdir():
    if x.is_file():
        counterFiles +=1
    elif x.is_dir():
        counterDir +=1
print("Num Files found: %d" %counterFiles)
print("Num Directories found: %d" %counterDir)
```


Modules

- **File-system operations**
 - Modules presented: `std library`, `os`, `pathlib`
- **CLI & process management**
 - Modules presented: **`sys`, `argparse`, `invoke`, `subprocess`**
- Time operations
 - Modules presented: `time`, `datetime`
- Math operations
 - Modules presented: `math`, `random`
- Data structures operations
 - Modules presented: `pickle`

**PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS**

SYS Module

Most used properties:

- **`sys.version`**: returns the current version of the interpreter.
- **`sys.stdin`**: contains input from the command line
- **`sys.argv`**: contains the list of command line arguments
- **`sys.stdout`**: contains output to be displayed on the console
- **`sys.stderr`**: contains the exceptions returned by Python
- **`sys.path`**: contains the list of directories that the interpreter will search for the required modules.

SYS module: it provides access to properties and functions for interacting with the Python interpreter.

SYS Module

SYS module functions	
<code>sys.exit([args])</code>	Raise an exception, signaling an intention to exit the interpreter.
<code>sys.setrecursionlimit()</code>	Set the maximum depth of the recursive calls
<code>sys.getrecursionlimit()</code>	Get the maximum depth of the recursive calls
<code>sys.setdefaultencoding()</code>	Set the character encoding for string variables
<code>sys.getdefaultencoding()</code>	Get the current character encoding for string variables
<code>sys.settrace(<i>tracefunc</i>)</code>	Set the system's trace function, allowing the user to implement a Python source code debugger in Python

SYS Module

```
import sys
print("Current Python interpreter version "+sys.version)
```

```
$ Current Python interpreter version 3.8.12 | packaged by conda-forge
```

```
import sys
numArgs = len(sys.argv)
print("Total arguments passed:", numArgs)
#First argument is the name of the Python script
print("Name of Python script: ", sys.argv[0])
for i in range(1, numArgs):
    print(" Argument "+sys.argv[i])
```

SYS Module

Read data from the standard input

```
import sys
age = input("Insert your age")
print("Insert your message")
data = sys.stdin.readline()
#Reading exactly 5 characters
data5 = sys.stdin.readline(5)
```

```
$ Insert your age:
Insert your message:
```

Argparse Module

Argparse: parsing module for command-line Python interfaces

Example: the head Unix shell command

```
$ head file.txt
```

POSITIONAL ARGUMENT(s)

```
$ head -h
```

```
usage: head [-n lines | -c bytes] [file ...]
```

HELPER FUNCTION(s)

```
$ head -n 10 students.csv
```

OPTIONAL PARAMETER(s)

<https://docs.python.org/3/library/argparse.html>

Argparse Module

Build an empty parser:

```
import argparse
parser = argparse.ArgumentParser()
args = parser.parse_args()
```

argp.py

```
$ python3 argparse_example.py
```

```
$ python3 argparse_example.py -h
usage: argparse_example.py [-h]
optional arguments:
  -h, --help  show this help message and exit
```

Argparse Module

Add a **POSITIONAL** argument:

```
ArgumentParser.add_argument(name or flags... [, action]  
[, nargs] [, const] [, default] [, type] [, choices]  
[, required] [, help] [, metavar] [, dest])
```

- name: name of the argument
- type - the type to which the argument should be converted
- help - a brief helper description of the argument
- nargs - the number of command-line arguments that should be consumed
- ...

Argparse Module

Add a **POSITIONAL** argument:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("value", help="value to be converted",
                    type=int)
args = parser.parse_args()
```

argp.py

```
$ python3 argparse_example.py 10
```

Argparse Module

Add a **OPTIONAL** argument:

```
ArgumentParser.add_argument(name or flags... [, action]  
[, nargs] [, const] [, default] [, type] [, choices]  
[, required] [, help] [, metavar] [, dest])
```

- name: name of the argument
- type - the type to which the argument should be converted
- help - a brief helper description of the argument
- nargs - the number of command-line arguments that should be consumed
- ...

Argparse Module

Add an **OPTIONAL** argument:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("value", help="value to be converted",
type=int)
parser.add_argument("-b", "-base", help="base value",
type=int)
args = parser.parse_args()
print(args.b)
```

argp.py

```
$ python3 argparse_example.py 10 -b 2
$ 2
```

Argparse Module

Add an **OPTIONAL** argument (**MULTIPLE** arguments):

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-b", "-base", help="base value",
nargs='+')
args = parser.parse_args()
print(args.b)
```

argp.py

```
$ python3 argparse_example.py -b 2 4 6 8
$ [2,4,6,8]
```

Argparse Module

Add an **OPTIONAL** argument (**NO** arguments):

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-b", "--base", help="base value",
                    action="store_true")
args = parser.parse_args()
print(args.b)
```

argp.py

```
$ python3 argparse_example.py -b
$ True
```

Invoke Module

- **Task:** a portion of Python code which can get some CLI arguments in input, do some computation and produce some outputs.
- Through the Invoke module API, it is possible to:
 - Define a task
 - Parse the input parameters
 - Execute a single task, and/or define a chain of tasks to be executed, in a sequential way and/or define a pool of parallel tasks

Invoke module: it provides facilities to create and execute Command Line Interface (CLI) Python tasks.

Invoke Module

Basic command line layout:

```
$invoke task1 [--task1_opt] .... taskN [--taskN_opt]
```

```
$invoke -- list
```

- Tasks are defined in the Python file named tasks.py (default)
- Each task is a function with a **decorator** (@task)
- Task parameters are provided via the CLI
- Task can execute local shell commands through the run(cmd) function
- Every task must have at least one parameter, which contains the system context (e.g. global variables and configuration)

Invoke Module

```
from invoke import task
@task()
def hello(c):
    print("Hello world")
@task()
def sum(c,x,y):
    r = int(x) + int(y)
    print(r)
```

tasks.py

```
$invoke sum -x 3 -y 4
7
```

```
$invoke sum 3 4
7
```


Invoke Module

```
from invoke import task
@task()
def hello(c):
    print("Hello world")
@task()
def sum(c,x,y):
    r = int(x) + int(y)
    print(r)
```

tasks.py

```
$invoke -l
hello
sum
```

Invoke Module

```
from invoke import task, run
```

```
@task()
```

```
def after(c):
```

```
    run("date")
```

```
@task
```

```
def before(c):
```

```
    run("date")
```

```
@task(pre=[before], post=[after])
```

```
def sleep(c):
```

```
    time.sleep(5)
```

tasks.py

Executes task sleep
after tasks before and sleep

```
$ invoke sleep
```

```
Mar 22 Nov 2022 17:46:37 CET
```

```
Mar 22 Nov 2022 17:46:42 CET
```

Subprocess Module

The subprocess module allows to spawn new processes, connect to their input/output/error pipes, and obtain their return codes.

```
subprocess.run(args, *, stdin=None, input=None, stdout=None,  
stderr=None, capture_output=False, shell=False, cwd=None,  
timeout=None, check=False, encoding=None, errors=None, text=None,  
env=None, universal_newlines=None, **other_popen_kwargs)
```

Subprocess Module

```
import subprocess

try:

    #Returns the list of active processes
    result = subprocess.run(
        ["ls", "-l"],
        text=True,
        capture_output=True
    )

    print(result.stdout.split())

except subprocess.CalledProcessError as e:
    print("Command failed:", e)
```

Modules

- **File-system operations**
 - Modules presented: std library, os, pathlib
- **CLI & process management**
 - Modules presented: sys, argparse, invoke, subprocess
- **Time operations**
 - Modules presented: **time, datetime**
- **Math operations**
 - Modules presented: math, random
- **Data structures operations**
 - Modules presented: pickle

**PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS**

Time Module

- Other time-related functionalities provided by the datetime module.
- The **epoch** is the point where the time starts. By convention, the time starts at January 1, 1970, 00:00:00 (UTC) on all platforms.
- GMT refers to Greenwich Mean Time (same as UTC).
- Most functions works with **struct_time** data structure:
 - tm_year, es. 1978
 - tm_mon: [1,12]
 - tm_mday: [1,31]
 - tm_hour, tm_min, tm_sec, tm_wday, tm_yday

Time module provides time-related functionalities

Time Module

- `time()`: returns the number of seconds elapsed since epoch
- `ctime()`: returns the local time as a string
- `localtime()`: returns the local time as a `struct_time`
- Built-in functions for time conversion:

From	To	Use
seconds since the epoch	<code>struct_time</code> in UTC	<code>gmtime()</code>
seconds since the epoch	<code>struct_time</code> in local time	<code>localtime()</code>
<code>struct_time</code> in UTC	seconds since the epoch	<code>calendar.timegm()</code>
<code>struct_time</code> in local time	seconds since the epoch	<code>mktime()</code>

Time Module

```
import time
print("Seconds elapsed since epoch: %f" %time.time())
print("Year %d" %time.localtime().tm_year)
print("Month %d" %time.localtime().tm_mon)
print("Day %d" %time.localtime().tm_mday)
```

```
$Seconds elapsed since epoch: 1668974665.144385
Year 2022
Month 11
Day 20
```


Time Module

`sleep(X)`: suspends the execution of the current thread for X seconds

```
import time
print("Hello timer started")
time.sleep(5)
print("Hello timer ended")
```

```
$Hello timer started
Hello timer ended
```

DateTime Module

- Allows time-related math operations (e.g. time difference)
- Supported classes:
 - `datetime.date`: year, month, day
 - `datetime.time`: hour, minute, second, microsecond, tz
 - `datetime.datetime`: combination of date + time
 - `datetime.timedelta`: differences between two datetimes
 - `datetime.timezone`: fixed offset from UTC
 - Get current time: `datetime.date.today()`,
`datetime.datetime.today()`

Datetime module: it provides functions for date/time manipulation

DateTime Module

```
import datetime

print("Today is: ",datetime.date.today())
print("Today is: ",datetime.datetime.today())
print("Year: ",datetime.datetime.today().year)
```

```
$Today is: 2022-11-21
Today is: 2022-11-21 13:12:26.457889
Year: 2022
```

DateTime Module

```
import datetime

tS = datetime.date(year = 2022, month = 11, day = 23)
tE = datetime.date(year = 2022, month = 11, day = 19)
tD = tS - tE
print("Date difference =", tD, type(tD))

tS = datetime.datetime(year = 2022, month = 11, day = 12, hour = 8, minute =
0, second = 40)
tE = datetime.datetime(year = 2022, month = 10, day = 7, hour = 7, minute = 0,
second = 10)
tD = tS - tE
print("Datetime difference =", tD, type(tD))
```

```
$Date difference = 4 days, 0:00:00 <class 'datetime.timedelta'>
Datetime difference = 36 days, 1:00:30 <class 'datetime.timedelta'>
```

Modules

- **File-system operations**
 - Modules presented: `std library`, `os`, `pathlib`
- **CLI & process management**
 - Modules presented: `sys`, `argparse`, `invoke`, `subprocess`
- **Time operations**
 - Modules presented: `time`, `datetime`
- **Math operations**
 - Modules presented: `math`, `random`
- **Data structures operations**
 - Modules presented: `pickle`

**PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS**

Math Module

Math constants:

- `math.pi`: 3.141592...
- `math.e`: 2.718281...
- `math.tau`: $2 * \text{math.pi} = 6.283185...$
- `math.inf`: floating-point positive infinity
- `math.nan`: floating-point “not a number” (NaN) value. Check whether a number is nan with the `isnan()` function.

`Math` module: it provides built-in mathematical functions and constants

<https://docs.python.org/3/library/math.html>

Math Module

Examples of math functions

<code>math.cos(x)</code>	Returns the cosine of number x
<code>math.sin(x)</code>	Returns the sin of number x
<code>math.tan(x)</code>	Returns the tangent of number x
<code>math.pow(x,y)</code>	Returns the value of x to the power of y
<code>math.log(x)</code>	Returns the natural logarithm of number x
<code>math.log2(x)</code>	Returns the base-2 logarithm of number x
<code>math.sqrt(x)</code>	Returns the square root of number x
<code>math.floor(x)</code>	Returns the nearest integer to x
<code>math.ceil(x)</code>	Rounds a number up to the nearest integer of x

Math Module

```
import math
num = 36
print("Square root: %f" %math.sqrt(num))
print("2-base Logarithm: %f" %math.log2(num))
print("Sin: %f" %math.sin(num))
print("Cosin: %f" %math.cos(num))
print("Square: %f"%math.pow(num,2))
```

```
$Square root: 6.000000
2-base Logarithm: 5.169925
Sin: -0.991779
Cosin: -0.127964
Square: 1296.000000
```


Modules

- **File-system operations**
 - Modules presented: std library, os, pathlib
- **CLI & process management**
 - Modules presented: sys, argparse, invoke, subprocess
- **Time operations**
 - Modules presented: time, datetime
- **Math operations**
 - Modules presented: math, random
- **Data structures operations**
 - Modules presented: **pickle**

PYTHON
MODULES THAT
WILL BE
PRESENTED
IN THIS CLASS

Pickle Module

- Pickling is equivalent to serializing or flattening.
- Serialized Python objects can be saved on disk, then loaded and deserialized again into a variable.
- It differs from JSON encoding/decoding since pickle module uses a binary serialization format (while JSON uses text)
- The serialization protocol is Python-specific and not universal. There are six versions available (based on Python language version).

Pickle module: it allows to serialize and deserialize Python objects

Pickle Module

- **`pickle.dump(data, file, version)`**
 - `data` is the Python data structure to be converted into a byte stream.
 - `file` is the name of the file where to save the stream.
 - `version` is the optional parameter related to the protocol's version
- **`pickle.load(file)`**
 - `file` is the name of the file containing the pickled data.
 - Returns the reconstituted object hierarchy specified in the file.
 - The protocol version is automatically detected from file.

Pickle Module

```
import pickle
file = open('myfile', 'wb')
data = ["Test", "Hello", "Let's try"]
pickle.dump(data, file)
file.close()
file = open('myfile', 'rb')
data = pickle.load(file)
file.close()

for item in data:
    print("Current item", item)
```